

SERVIÇOS WEB

Microserviços com Molecular

Arquitetura, padrões e laboratório prático

IFSul – Campus Passo Fundo

O que vamos cobrir hoje

01

Desafios reais de microserviços

Por que "só Docker" não basta?

02

Padrões arquiteturais fundamentais

Discovery, LB, Circuit Breaker, Gateway, Saga...

03

Molecular em profundidade

Transporters, serialização, fault tolerance

04

Framework vs. Service Mesh

Molecular × Istio × Linkerd

05

Mini-laboratório ao vivo

3 serviços + NATS + Redis + Docker Compose

Por que microsserviços são difíceis?



Service Discovery

Como serviço A encontra o endereço de B?



Falhas em cascata

Timeout em B paralisa A, C, D...



Latência de rede

Cada chamada atravessa a rede (ms contam)



Consistência de dados

Sem transação ACID entre serviços



Deploy coordenado

Versões incompatíveis quebrando em produção



Observabilidade

Rastrear um bug em 10 serviços é um pesadelo

Padrões arquiteturais fundamentais

Service Discovery

Registry dinâmico: serviços anunciam e consultam endereços automaticamente (DNS interno, Consul, etcd).

Load Balancing

Distribuição de carga entre instâncias: round-robin, least-connections, hash consistente.

Circuit Breaker

Abre o circuito após falhas repetidas, evitando cascata. Estados: Closed → Open → Half-Open.

API Gateway

Ponto único de entrada externo: autenticação, rate limiting, roteamento e tradução de protocolo.

Event-Driven

Comunicação assíncrona via eventos (pub/sub). Produtores e consumidores são desacoplados.

Saga Pattern

Coordena transações distribuídas via sequência de eventos compensáveis — sem 2PC distribuído.

Moleculer

Framework de microsserviços para Node.js — progressivo, rápido e tolerante a falhas

Service Registry

Cada nó anuncia suas ações no broker; descoberta automática

Caching integrado

Redis, Memcached ou in-memory; invalidação por chave/grupo

Retry & Fallback

Políticas declarativas: retries com backoff exponencial

Transporte plugável

NATS, Redis, AMQP, Kafka, TCP, NATS Streaming, gRPC

Circuit Breaker

Built-in, configurável por ação com estados e callbacks

Validação

Fastest-validator integrado — schemas declarativos por ação

Transporters & Serialização

Transporters

- **NATS**

Recomendado — leve, alta performance, pub/sub nativo

- **Redis**

Bom para quem já usa Redis (cache + transporte)

- **AMQP / RabbitMQ**

Filas persistentes, dead-letter, roteamento avançado

- **Kafka**

Volume extremo, replay de mensagens, log compaction

- **TCP**

Sem dependência externa — P2P direto entre nós

- **gRPC**

Comunicação binária tipada entre linguagens

Serialização

JSON

Padrão — legível, compatível

MessagePack

Binário, ~2× menor que JSON

Protobuf

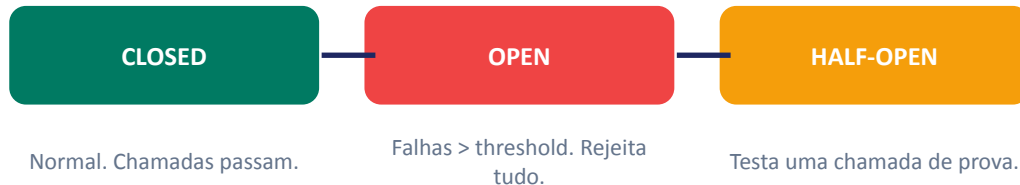
Schema-first, tipado, muito rápido

Avro

Bom para Kafka + Schema Registry

Fault Tolerance no Moleculer

Circuit Breaker — estados



Configuração no código:

```
circuitBreaker: {  
  enabled: true,  
  threshold: 0.5,    // 50% de falhas  
  windowTime: 60,   // janela 60s  
  halfOpenTime: 10000  
}
```

Outras proteções

Retry

Backoff exp.
config. por ação

Timeout

Limite de espera
por chamada

Bulkhead

Limita chamadas
simultâneas

Fallback

Resposta padrão
em caso de falha

Framework vs. Service Mesh

Molecular / Spring Cloud

Istio / Linkerd / Cilium

Camada	Código da aplicação	Infraestrutura (proxy sidecar)
Linguagem	Específico (Node/Java/Go)	Agnóstico — qualquer linguagem
Curva de aprendizado	Moderada	Alta (K8s obrigatório)
Latência extra	Nenhuma	~1-2 ms por hop (sidecar)
Observabilidade	Manual / por biblioteca	Automática (traces, metrics)
mTLS / segurança	Manual	Automático + zero-trust
Ideal para	Times pequenos, Node.js	Multi-linguagem, K8s, escala